"Advanced Natural Language Processing (NLP): Transformers. Attention is All You Need."

Nazarenko Elena, PhD

Bio

Elena Nazarenko, PhD

- Senior Lecturer at HSLU/ Co-head of NLP/LLMs bootcamp
- Senior Data scientist/NLP developer and AI expert at "BIAS" project.
- Served as a Head of Data and AI at Witty Works.
- Built a core algorithm of Witty - inclusive writing assistant at Witty works (part of Hugging Face start-up accelerator, Finalist of Microsoft’s Entrepreneurship for Positive Impact Cup 2024)

Background:

PhD University Grenoble Alpes, Research institutes in France, Sweden, Switzerland (Paul Scherrer Institute - ETH Domain)

Agenda. Day 1

- Introduction to NLP
- Foundational Principles of NLP
- What are Vector Embeddings? Understanding the Foundation of Data Representation
- Hands-on: Named Entity Recognition
- Introduction to Large Language Models (LLMs)
- Transformers Architecture
- Hands-on Visualising Transformers
- Hands-on HuggingFace Transformers
- Itro Sentence Transformers
- Hands-on on Semantic Similarity
- Hands-on Zero-shot classification with SetFit
- Prompt Engineering
- Hands-on: Prompt engineering

Introduction to NLP

Definition

Natural Language Processing (NLP) is a branch of computer science and artificial intelligence that focuses on the interaction between computers and human language.

Aim

NLP aims to enable computers to understand, interpret, and generate human language in a way that is both meaningful and useful.

Technical Goal

NLP empowers machines to process and analyze vast amounts of textual data, unlocking valuable insights and enabling efficient communication.

Challenges

NLP faces challenges like ambiguity in language, variations in dialects and accents, and the need for robust algorithms to handle complex linguistic structures.

The Evolution of NLP: From Rule-Based Systems to Deep Learning and Transformers

1. ### Rule-Based Systems (1950s-1980s)

   Early NLP systems relied on handcrafted rules and linguistic patterns to process language, requiring extensive knowledge of grammar and syntax.

2. ### Statistical NLP (1990s-2010s)

   The rise of machine learning brought about statistical NLP, which leverages statistical models trained on large datasets to identify patterns and make predictions about language.

3. ### Deep Learning Era (2010s-Present)

   Deep learning, particularly recurrent neural networks (RNNs) and transformers, has revolutionized NLP, enabling models to learn complex language representations and achieve state-of-the-art performance on various tasks.

4. ### Transformers and Pretrained Models (2018-Present)

   Introduction of Transformer architecture by Vaswani et al. (2017).
   Key Models: BERT (Bidirectional Encoder Representations from Transformers), GPT (Generative Pre-trained Transformer), T5, RoBERTa.
   Features: Self-attention mechanism, scalability, and pretraining on massive datasets.
   Impact: Significant improvements in a wide range of NLP tasks, state-of-the-art performance.

Unlocking Meaning: Core Concepts in NLP

At its core, NLP centers around breaking down human language into its fundamental components and understanding the relationships between them. Key concepts include:

Text Processing

The foundation of NLP involves cleaning and preparing raw text data, such as removing punctuation, converting to lowercase, and handling special characters, to ensure consistency and improve model performance.

Tokenization

The process of dividing text into individual words or tokens, which serve as the building blocks for further analysis. Tokenization can be as simple as splitting text by spaces or involve more complex rules for handling contractions, punctuation, and special characters.

Stemming & Lemmatization

Techniques for reducing words to their base or root form, such as "running" to "run" or "better" to "good," which helps to consolidate words with similar meanings and reduce vocabulary size, improving model efficiency.

Introduction to NLP. What can be done with NLP

Text summarization

Named entities recognition

Part-of-Speech Tagging
Semantic dependencies

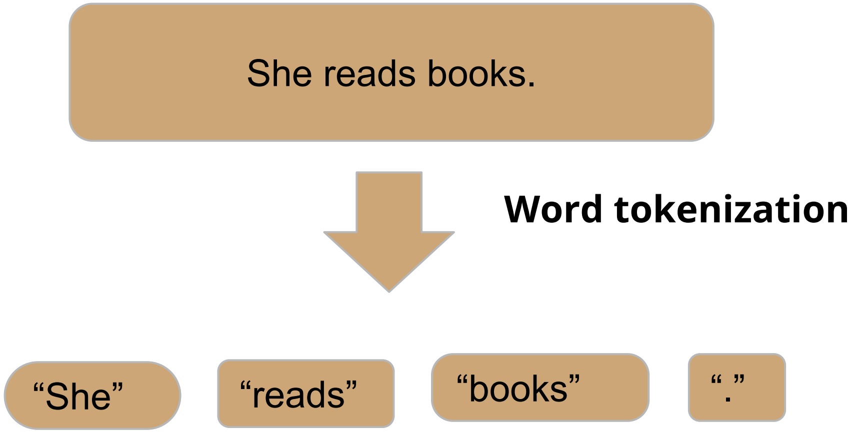
Sentiment Analysis

Text classification

Text Generation

Foundational Principles of NLP - I

Tokenization-first step: splitting text into smaller chunks, typically words or subwords.



Tokenization in Natural Language Processing

Tokenization transforms text into numerical data, crucial for machine learning models in NLP.

The process includes several stages:

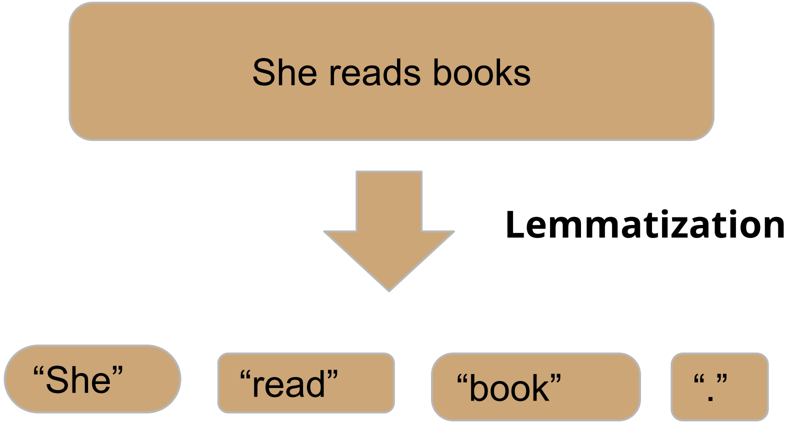
- **Normalization:** Ensures text consistency by cleaning it, like converting letters to lowercase, removing punctuation, or altering accented characters based on the task specifics. While more normalization simplifies data, excessive cleaning can lose context.
- **Pre-tokenization:** Splits the text into segments, such as words, especially when spaces separate them. However, this varies by language.
- **Token Cleanup:** Involves cleaning steps after initial splitting. This might include removing stopwords like "a" or "the," as they add little value to the task at hand.

Balance of Complexity and Context

- **Simple Models:** Require extensive data cleaning to reduce complexity, especially with small datasets.
- **Advanced AI Systems:** Tools like OpenAI's ChatGPT and Google's Gemini retain complexity for nuanced understanding, often leveraging subword tokenization for detailed language analysis.
- **Key Takeaway:** Balance simplicity with essential information to optimize machine learning model training.

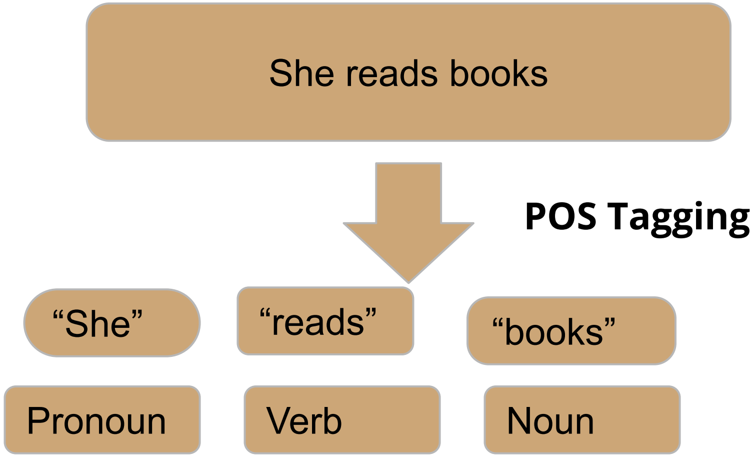
Foundational Principles of NLP - I

Lemmatization: it is a process to change the word to its dictionary form, known as the “lemma”.



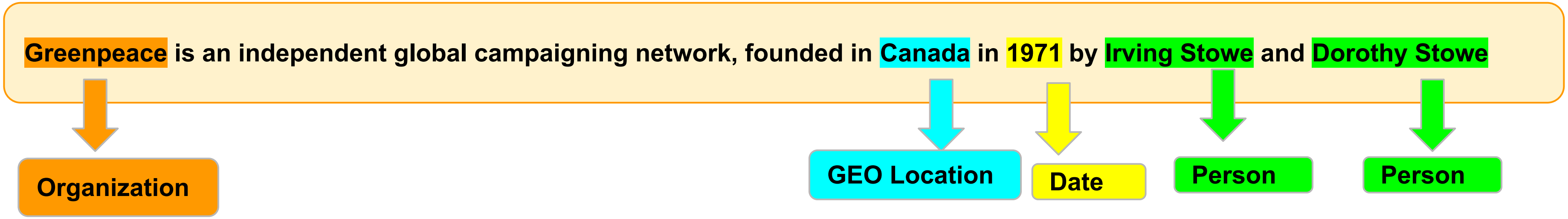
Foundational Principles of NLP - I

Part-of-Speech (POS) Tagging : it is a process of labeling each word (token) in a text with its corresponding part of speech



Foundational Principles of NLP - II

Named Entity Recognition: information extraction from the text that classifies named entities into predefined categories such as the person names, organizations, locations, numbers, specific terms (medical, legal), percentages, etc.



Importance:
Information Retrieval: Helps in improving the accuracy of search systems by focusing on key terms.
Question Answering: Enables systems to answer questions related to specific entities.
Content Recommendation: Helps in content personalization and recommendation by focusing on key entities.
Research: Useful in extracting structured information from massive datasets for academic or corporate research.

Foundational Principles of NLP - II

Sentiment Analysis is a task to determine and extract the sentiment or emotion in the text. The primary goal to detect positive/negative/neutral tone of text.

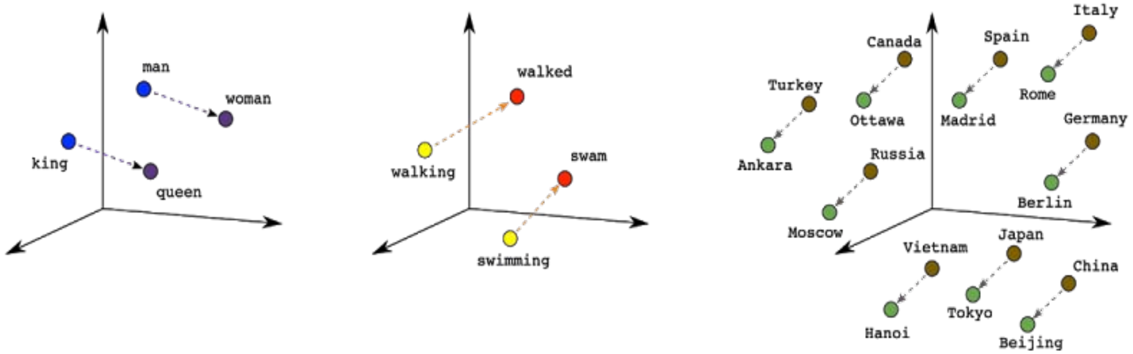
Types:

- **Binary Sentiment Analysis:** Categorizes sentiments as positive or negative.
- **Fine-grained Sentiment Analysis:** Goes beyond binary and might include categories like very positive, positive, neutral, negative, and very negative.
- **Emotion detection:** Determines specific emotions being expressed, e.g., happiness, anger, sadness, etc.

What are Vector Embeddings? Understanding the Foundation of Data Representation

- Embeddings are representations of data as points in space
- Any type of data can be embedded: text, images, videos, users, music
- The key: locations in this space are semantically meaningful
- Think of it as converting data into coordinates with meaning

What are Vector Embeddings? Understanding the Foundation of Data Representation



Male-Female

Verb Tense

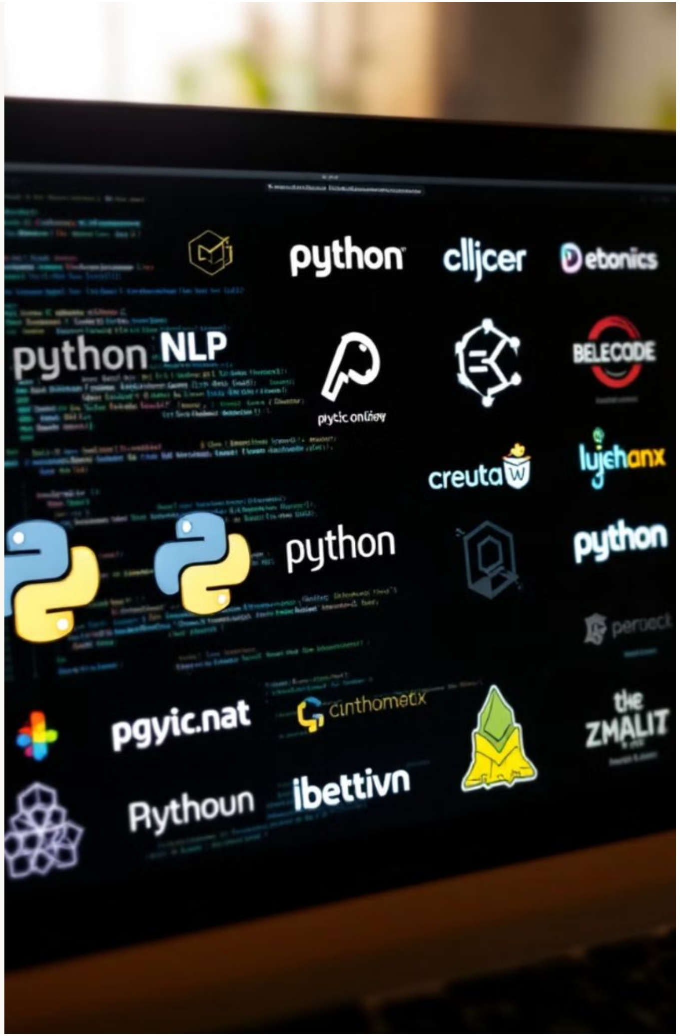
Country-Capital

Embeddings are a way of representing data–almost any kind of data, like text, images, videos, users, music, whatever–as points in space where the locations of those points in space are semantically meaningful.

[Word2Vec]

NLP Toolkit Ecosystem

Library	Primary Use	Best For
NLTK	Academic research	Learning, experimentation
SpaCy	Production applications	Speed, multilingual support (60+ languages)
Stanza	Research-grade analysis	Accuracy, linguistic detail
TextBlob	Simplified tasks	Quick prototyping, beginners
Gensim	Topic modeling	Document similarity, large corpora



Hands-on
Named Entity Recognition

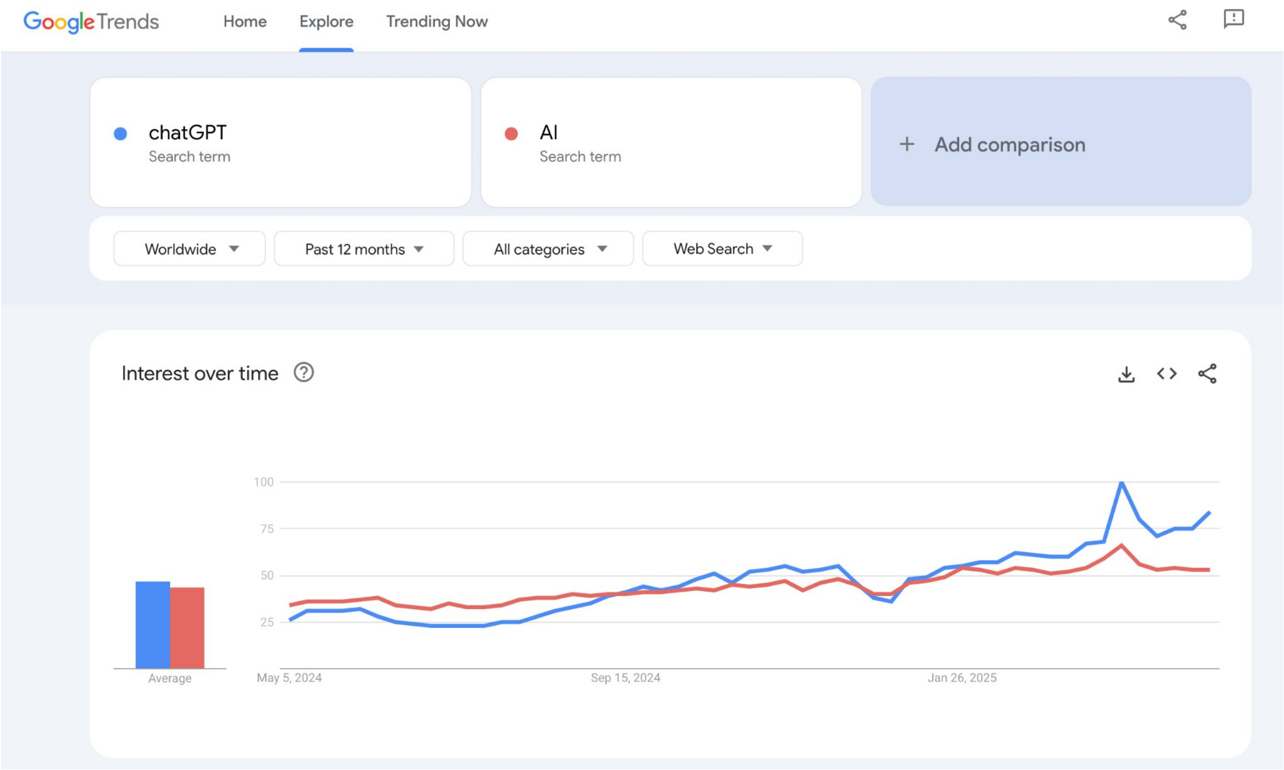
Intro to LLMs. Transformers

Andrej Karpathy
@karpathy

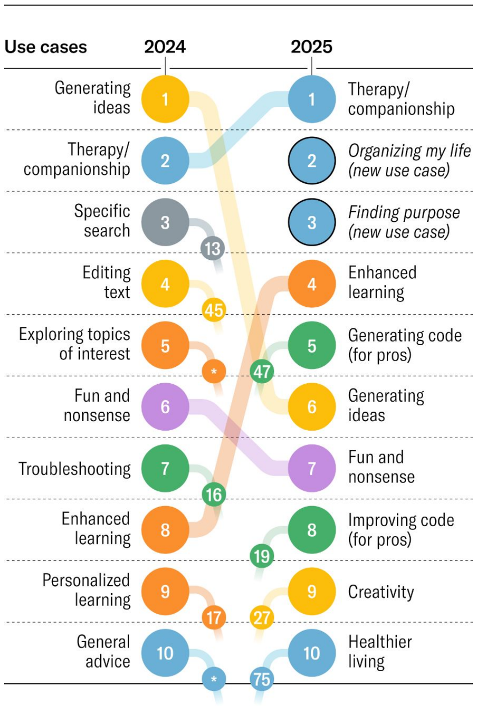
The hottest new programming language is English

12:14 PM · Jan 24, 2023 · **3.8M** Views

User interest has doubled in the last 12 months



Top 10 Gen AI Use Cases



Top 10 Gen AI Use Cases

The top 10 gen AI use cases in 2025 indicate a shift from technical to emotional applications, and in particular, growth in areas such as therapy, personal productivity, and personal development.

Themes

- PERSONAL AND PROFESSIONAL SUPPORT
- TECHNICAL ASSISTANCE AND TROUBLESHOOTING
- CONTENT CREATION AND EDITING
- CREATIVITY AND RECREATION
- LEARNING AND EDUCATION
- RESEARCH, ANALYSIS, AND DECISION-MAKING

Harvard Business Review
https://hbr.org/2025/04/how-people-are-really-using-gen-ai-in-2025

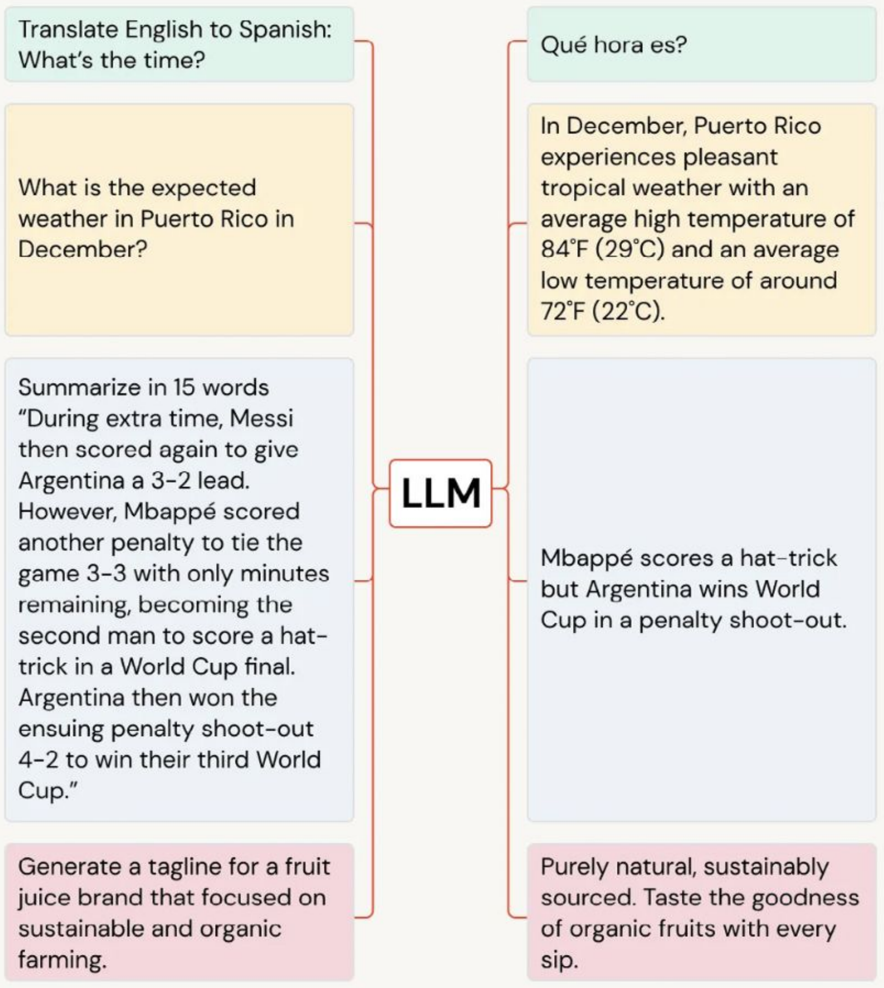
What exactly is an Large Language Models?

Simple Analogy:

Think of an LLM as a super-smart autocomplete on your phone. It's been fed massive amounts of text and code, and it's learned to predict what might come next.

More Technical:

A "large language model" is essentially a type of computer program adept at understanding and generating human language. It's trained on vast datasets consisting of text and code, enabling it to predict and generate coherent language based on the input it receives.



Large Language Models

A Large Language Model (LLM) is a machine learning model designed to handle and generate human-like text based on vast amounts of training data and trained.
Fondation Models

LLMs in Action: Cool Things They Can Do

1 Natural Language Understanding

LLMs excel at understanding and interpreting human language. They can comprehend context, infer meaning, and provide relevant responses to queries.

2 Text Generation

These models are capable of generating coherent and contextually relevant text. From writing articles to composing poetry, LLMs can produce content in various styles and genres.

3 Language Translation

LLMs facilitate seamless language translation by converting text from one language to another while preserving context and meaning.

4 Code Generation

With their understanding of programming languages, LLMs can assist developers by generating code snippets, automating repetitive tasks, and offering solutions to coding challenges.

5 Creative Collaboration

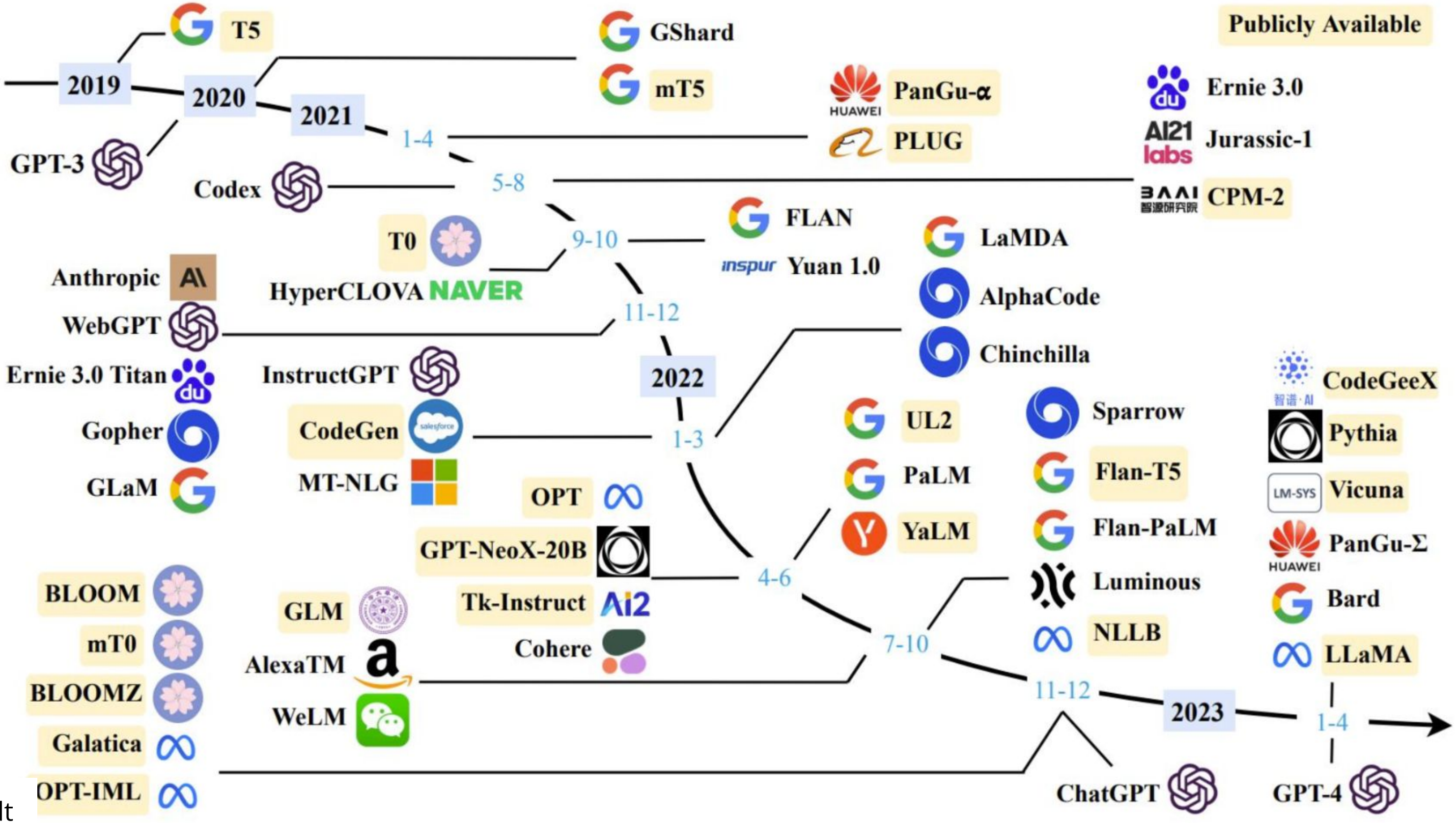
LLMs can serve as collaborative partners in brainstorming sessions, suggesting ideas, refining concepts, and providing inspiration across diverse fields such as art, design, and storytelling

6 Overall: Increase productivity!

LLMs can help boost productivity by automating tasks, generating content, and providing valuable insights.

https://arxiv.org/abs/2303.18223

LLMs since GPT-3



by Philip Schmidt

"Attention is All You Need."

by Vaswani et al. in 2017

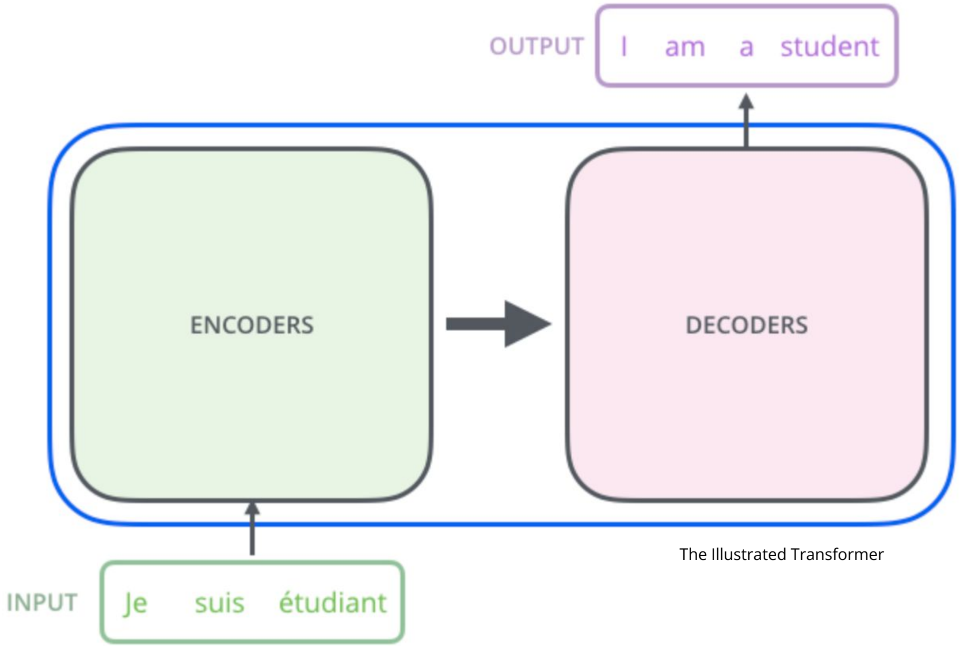
Transformers & Attention Mechanism:

Transformers - new architecture, which relies entirely on self-attention mechanisms.This enables the model to capture relationships in the data regardless of the distance between elements.

She poured water from the pitcher to the cup until it was full.

She poured water from the pitcher to the cup until it was empty.

Transformer architecture



The Illustrated Transformer

Encoder (left): The encoder receives an input and builds a representation of it (its features). This means that the model is optimized to acquire understanding from the input.
Decoder (right): The decoder uses the encoder's representation (features) along with other inputs to generate a target sequence. This means that the model is optimized for generating outputs.

Transformers models

Encoder-only models: Good for tasks that require understanding of the input, such as sentence classification and named entity recognition. (BERT-family)

Decoder-only models: Good for generative tasks such as text generation. (GPT-family, Llama-family, Google PaLM-family)

Encoder-decoder models or sequence-to-sequence models: Good for generative tasks that require an input, such as translation, summarization, question-answering (Google T5)

https://poloclub.github.io/transformer-explainer/

Hands-on Visualising Transformers

HuggingFace Transformers

https://huggingface.co/

Itro Sentence Transformers

Hands-on on Semantic Similarity

Hands-on on Zero-shot classification with SetFit

Sources to learn more about transformers

How do Transformers work?
https://huggingface.co/learn/nlp-course/chapter1/4

The Illustrated Transformer
http://jalammar.github.io/illustrated-transformer/

Transfer Learning

- Identify the task type for your business problem
- Pick and test a pre-trained model

  -No need to prepare a large dataset

  -Less coding

- Optionally, fine-tune the model on your data

  -Much less data is required

  -No need to train for long periods of time

Which model to choose?

Search models, datasets, users...

Models 1,669,014

Filter by name

Full-text search

Add filters   Sort: Trending

nvidia/parakeet-tdt-0.6b-v2
Automatic Speech Recognition • Updated 6 days ago • 20.4k • 412

deepseek-ai/DeepSeek-Prover-V2-671B
Text Generation • Updated 7 days ago • 4.98k • • 711

nari-labs/Dia-1.6B
Text-to-Speech • Updated 1 day ago • 136k • • 1.91k

JetBrains/Mellum-4b-base
Text Generation • Updated about 5 hours ago • 1.89k • 259

microsoft/Phi-4-reasoning-plus
Text Generation • Updated about 21 hours ago • 6.35k • 216

Qwen/Qwen3-235B-A22B
Text Generation • Updated 7 days ago • 53.6k • • 713

1 660 014 models!

https://huggingface.co/models

Large Language Models. Best practices

Achieve best possible performance

- Use best models (could be LLM API)
- Use prompts with detailed task context, relevant information, instructions
- Experiment with prompt engineering techniques
- Experiment with few-shot examples that are 1) relevant to the test case, 2) diverse (if appropriate)
- Spend quality time optimizing a pipeline / "chain"

Optimize Cost

- Start optimizing for cost once you reach your expected performance
- Use techniques like Quantization, Distillation or smaller models trained on synthetic data

Prompt engineering

You don’t need to be a data scientist or a machine learning engineer – everyone can write a prompt.

What is a prompt?

A **prompt** is natural language text that tells the AI what task to perform.

In a text-to-text language model, a prompt can take many forms:

- A question or query
- A command
- A longer statement with context, instructions, or past conversation

Aspects of the Prompt Engineering

Accessible to Everyone

You don't need to be a data scientist or ML engineer - anyone can write effective prompts.

Iterative Process

Crafting effective prompts requires experimentation and refinement.

Multiple Factors

Model choice, training data, configurations, word choice, and context all impact prompt effectiveness.

What Does Prompt Engineering Involve?

Carefully phrasing the query to guide the model's response:

- Specifying tone, style, or structure (e.g., formal vs. casual)

- Choosing the right words and grammar

- Providing relevant context or background

- Describing a persona or role for the AI to adopt

➡️ The goal is to get more **accurate**, **reliable**, and **context-aware** outputs from the model.

LLM Output Configuration

Effective prompt engineering requires setting optimal configurations for your specific task.

Model Selection

Choose the appropriate model (Gemini, GPT, Claude, Gemma, LLaMA) for your specific needs.

Configuration Settings

Adjust parameters like temperature and token limits to control the output quality and style.

Output Format

Specify the desired format (text, JSON, code) to ensure usable results.

Output Length Considerations

Token Limits

Setting appropriate token limits is crucial for controlling response length.

- Higher limits allow more comprehensive responses
- Lower limits reduce costs and computation time

Prompt Engineering Impact

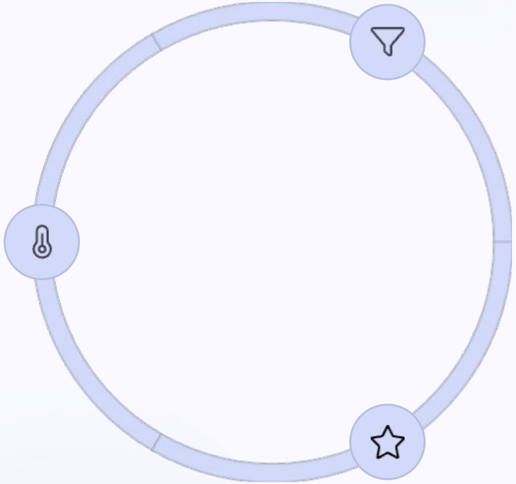
Output length restrictions require prompt engineering adjustments.

- Shorter outputs need more precise prompts
- Specific techniques like ReAct benefit from length control

Sampling Controls

Temperature

Controls randomness in token selection. Lower values (0-0.3) produce more deterministic responses, while higher values (0.7-1.0) create more diverse outputs.



Top-K

Selects the K most likely tokens from the model's predicted distribution. Higher values increase creativity, lower values improve factual accuracy.

Top-P

Selects tokens whose cumulative probability doesn't exceed P. Values range from 0 (greedy) to 1 (all tokens considered).

Temperature Control

Low Temperature (0-0.3)

Deterministic, consistent outputs

Medium Temperature (0.4-0.7)

Balanced creativity and coherence

High Temperature (0.8-1.0)

More random, diverse outputs

Top-K and Top-P Sampling

Top-K Sampling

Selects the K most likely tokens from the model's predicted distribution.

- Higher values: more creative outputs
- Lower values: more factual outputs
- Top-K of 1: equivalent to greedy decoding

Top-P (Nucleus) Sampling

Selects tokens whose cumulative probability doesn't exceed P.

- Values range from 0 (greedy) to 1 (all tokens)
- Adapts to the confidence of the model's predictions
- Often used in combination with Top-K

Configuration Warnings

Relevance Risk

Higher freedom (temperature, top-K, top-P) may generate less relevant text.

Repetition Loop Bug

Inappropriate settings can cause the model to get stuck repeating the same words or phrases.

Balance Required

Finding the optimal balance between determinism and randomness prevents issues at both extremes.

Prompting techniques

Prompt engineering. Tips

https://www.promptingguide.ai

1. Start Simple
2. The Instruction:

```
### Instruction ###
Translate the text below to Spanish:
Text: "hello!"
```

3. Specificity: Be very specific about the instruction and task you want the model to perform
4. Avoid Impreciseness

```
Use 2-3 sentences to explain the concept of prompt engineering to a high school student.
```

Zero-Shot Prompting

https://www.promptingguide.ai

Prompt:

```
Classify the text into neutral, negative or positive.
Text: I think the vacation is okay.
Sentiment:
```

Output:

```
Neutral
```

Few-Shot Prompting

https://www.promptingguide.ai

Prompt:

```
A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses
the word whatpu is:
We were traveling in Africa and we saw these very cute whatpus.
To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses
the word farduddle is:
```

Output:

```
When we won the game, we all started to farduddle in celebration.
```

Brown et al. 2020

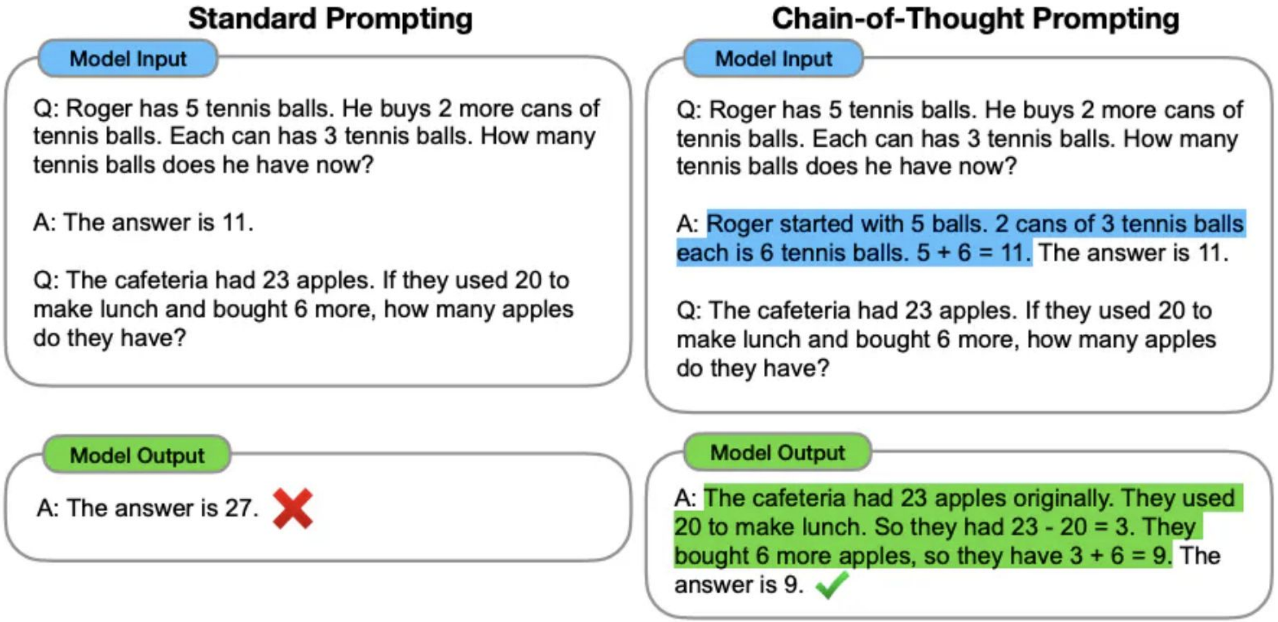
System, contextual and role prompting

- **System prompting** sets the overall context and purpose for the language model. It defines the ‘big picture’ of what the model should be doing, like translating a language, classifying a review etc.

- **Contextual prompting** provides specific details or background information relevant to the current conversation or task. It helps the model to understand the nuances of what’s being asked and tailor the response accordingly.

- **Role prompting** assigns a specific character or identity for the language model to adopt. This helps the model generate responses that are consistent with the assigned role and its associated knowledge and behavior.

Chain-of-Thought (CoT) Prompting



Wei et al. (2022)

Prompt-Chaining

Prompt-Chaining: Decomposes a task into sequential subtasks, where each step builds on previous results

ReAct (reason & act)

Prompt:

> Use Reasoning and Acting (ReAct) to solve this problem:
>
> - **"Thought:** Describe your reasoning."
> - **"Action:** (Options: Search, Do Nothing)"
> - **"Observation:** Report the result of the action."
>
> Repeat until you can answer.
>
> **Problem:** *What is the capital of Australia, and what's the current temperature there?*

ReAct (reason & act). Output

Thought: I know that the capital of Australia is Canberra, but I need to search for the current temperature.
Action: Search "current temperature in Canberra."
Observation: The current temperature in Canberra is 18°C.
Thought: I have both pieces of information now.
Final Answer: The capital of Australia is Canberra, and the current temperature there is 18°C.

Automatic Prompt Engineering

Step1. Write the prompt which will generate the output variants.

Prompt	`We have a band merchandise t-shirt webshop, and to train a chatbot we need various ways to order: "One Metallica t-shirt size S". Generate 10 variants, with the same semantics but keep the same meaning.`
Output	1. I'd like to purchase a Metallica t-shirt in size small. 2. Can I order a small-sized Metallica t-shirt? 3. I'd like to buy one Metallica t-shirt in small. 4. One Metallica shirt, size small, please. 5. I want one Metallica t-shirt in size small. 6. I'd like to place an order for one Metallica t-shirt in small. 7. Small Metallica t-shirt, one please. 8. I'm looking for one Metallica t-shirt in small. 9. Can you get me one Metallica t-shirt in small? 10. One Metallica small t-shirt, please.

Automatic Prompt Engineering

Step 2. Evaluation
Evaluate all of the instruction candidates by scoring the candidates based on a chosen metric: BLEU or ROUGE.

BLEU (Bilingual Evaluation Understudy):
Measures precision: How much of the generated text matches the reference.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):
Focuses on overlap between reference and generated text.
How much of the reference content appears in the generated output.

Hands-on
Prompt engineering

Natural Language Processing & Large Language Models

From foundations to LLM and beyond

A bootcamp for compact knowledge

Program Directors: Dr. Aygul Zagidullina and Dr. Elena Nazarenko

HSLU Lucerne University of Applied Sciences and Arts

Applied Sciences and Arts
Continuing Education

Bootcamp NLP & LLMs - Modules/ Days. **Starts September 19**

Day 1
Foundational Principles of NLP

- Introduction to NLP and preprocessing techniques
- Advanced NLP techniques
- Representation and embeddings
- Probabilistic and sequence models

Day 2
Transformer Models and Fundamentals of LLMs

- Introduction to transformer models
- Transfer learning
- Applying transformers for NLP tasks (chatbots, classification, summarization)
- Introduction to LLMs and prompt engineering

Day 3
Retrieval-Augmented Generation

- Introduction to vector databases
- RAG (LlamaIndex/LangChain)
- RAG variations (GraphRAG, CAG)
- Semantic search and document retrieval

Day 4
Language Agents and Model Evaluation in LLMs

- Fine-tuning (full and PEFT) vs. RAG (what to use when)
- Evaluation metrics for LLMs and RAG
- Bias in LLMs
- Responsible AI

Day 5
MLOps for NLP and Deploying Models to Production

- Introduction to MLOps for NLP and LLMs
- Deploying models for production
- Cloud vs. on-premise deployment strategies
- Testing strategies for AI applications

Day 6
Real-World Applications for NLP and LLMs

- Real-World Applications for NLP and LLMs
- Industry case studies
- Hands-on applied NLP tasks
- Emerging trends and challenges associated with LLMs